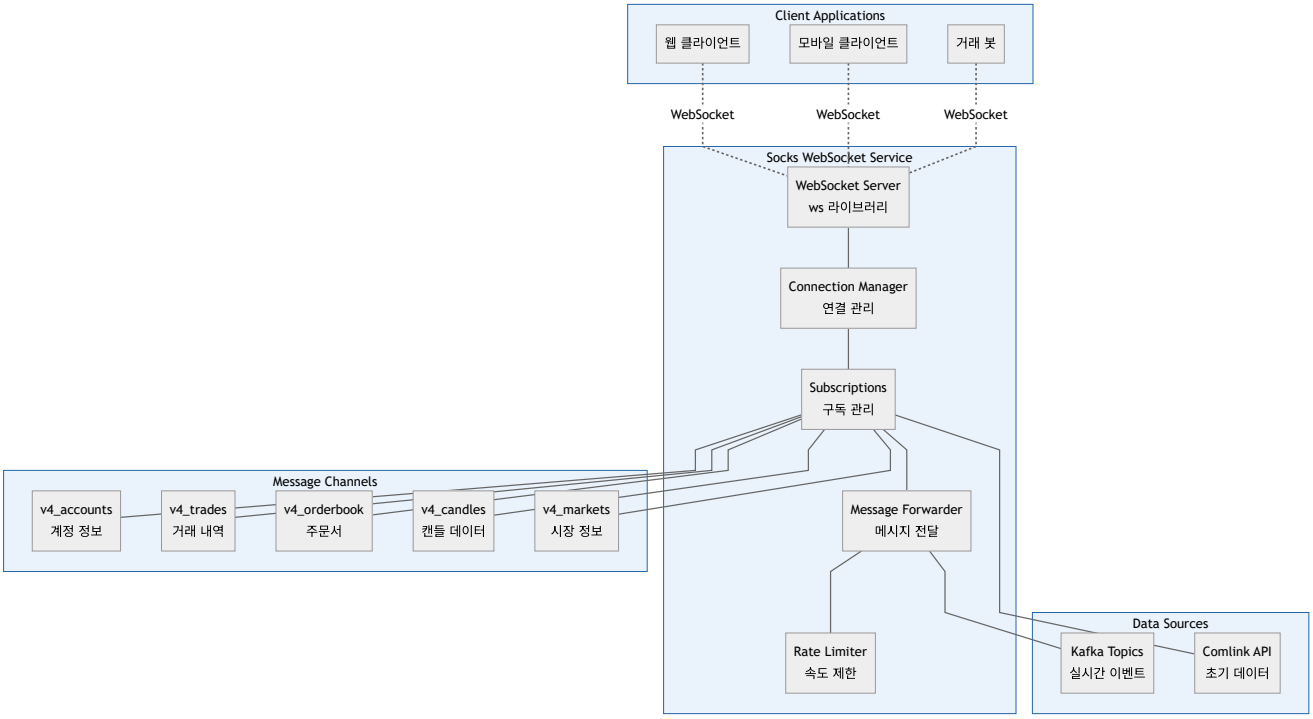
Socks WebSocket 서비스 구현 분석

개요

Socks는 dYdX v4 Indexer의 WebSocket 서비스로, 클라이언트에게 실시간 데이터 스트리밍을 제공합니다. Kafka에서 이벤트를 수신하여 구독 중인 클라이언트들에게 실시간으로 전달하는 역할을 담당하며, 다양한 채널(주문서, 거래, 계정, 시장 데이터 등)을 통해 데이터를 제공합니다.

아키텍처 개요



WebSocket 연결 관리 및 스케일링

1. WebSocket 서버 초기화

```
export class Wss {
private wss: WebSocket.Server;
private started: boolean;
private closed: boolean;

constructor() {
this.started = false;
this.closed = false;
this.wss = new WebSocket.Server({
port: config.WS_PORT,
});
}

public async start(): Promise<void> {
if (this.started) {
throw new Error('Wss already started');
}

this.started = true;

// 서버 에러 핸들링
this.wss.on(WebsocketEvents.ERROR, (error: Error) => {
```

```
logger.error({
at: 'wss#onError',
message: `WebSocket server threw error: ${error.message}`,
error,
});
});

// 서버 시작 대기
await new Promise((resolve) => {
this.wss.on(WebsocketEvents.LISTENING, resolve);
});
}

public onConnection(callback: (ws: WebSocket, req: IncomingMessage) => void): vo
this.wss.on(WebsocketEvents.CONNECTION, callback);
}
}
```

2. 연결 관리 시스템

```
export class Index {
// 연결 ID(UUID)와 WebSocket 연결 매핑
public connections: { [connectionId: string]: Connection };
```

```
private onConnection(ws: WebSocket, req: IncomingMessage): void {
const connectionId: string = uuidv4();


// 연결 정보 저장
this.connections[connectionId] = {
ws,
messageId: 0,
countryCode: getCountry(req),
};


const numConcurrentConnections: number = Object.keys(this.connections).length;
// 연결 통계 수집
stats.increment(`${config.SERVICE_NAME}.num_connections`, 1);
stats.gauge(`${config.SERVICE_NAME}.num_concurrent_connections`, numConcurrentCo


// 연결 확인 메시지 전송
try {
sendMessage(ws, connectionId, createConnectedMessage(connectionId));
} catch (error) {
logger.error({
message: `Failed to send connected message: ${error.message}`,
error,
connectionId,
```

```
});

}


// 이벤트 핸들러 등록

this.attachEventHandlers(ws, connectionId);

}

}
```

3. 하트비트 및 연결 상태 관리

```
private attachEventHandlers(ws: WebSocket, connectionId: string): void {

// 메시지 핸들러

ws.on(WebsocketEvents.MESSAGE, (message: WebSocket.RawData) => {

try {

this.onMessage(connectionId, message);

} catch (error) {

logger.error({

message: `Failed to handle message: ${error.message}`,

error,

connectionId,

});

sendMessage(

this.connections[connectionId].ws,
```

```
connectionId,
createErrorMessage('Internal error', connectionId, this.connections[connectionId
);
}
});


// 주기적 하트비트 시작
this.connections[connectionId].heartbeat = setInterval(
() => this.heartbeat(connectionId),
HEARTBEAT_INTERVAL_MS,
);


// Pong 응답 핸들러
this.connections[connectionId].ws.on(WebsocketEvents.PONG, () => {
// 연결 해제 타이머 취소
if (this.connections[connectionId].disconnect) {
clearTimeout(this.connections[connectionId].disconnect);
delete this.connections[connectionId].disconnect;
}
});


// 연결 종료 핸들러
this.connections[connectionId].ws.on(WebsocketEvents.CLOSE, (code: number, reaso
```

```
stats.increment(`${config.SERVICE_NAME}.num_disconnects`, 1, {
code: String(code),
reason: String(reason)
});
this.disconnect(connectionId);
});
}


private heartbeat(connectionId: string): void {
if (!this.connections[connectionId] || this.connections[connectionId].disconnect
return;
}
try {
// 타임아웃 후 연결 해제 예약
this.connections[connectionId].disconnect = setTimeout(
() => this.disconnect(connectionId),
HEARTBEAT_TIMEOUT_MS,
);
// Ping 전송
this.connections[connectionId].ws.ping();
} catch (error) {
logger.error({
message: `Heartbeat threw error: ${error.message}`,
```

```
error,
connectionId,
});
}
}
```

4. 안전한 메시지 전송

```
export function sendMessageString(
ws: WebSocket,
connectionId: string,
message: string,
): void {
// 연결 상태 확인
if (ws.readyState !== WebSocket.OPEN) {
stats.increment(`${config.SERVICE_NAME}.ws_message_not_sent`, {
reason: WEBSOCKET_NOT_OPEN,
readyState: ws.readyState.toString(),
});
return;
}
```

```
ws.send(message, (error) => {
if (error) {
const errorLog = {
at: 'wss#sendMessage',
message: `Failed to send message: ${error.message}`,
error,
connectionId,
};


if (error?.message.includes?.('write EPIPE')) {
// 원격 스트림 종료 - 정상적인 상황
logger.info(errorLog);
} else if (error?.message.includes?.('write ECONNRESET')) {
// 클라이언트 갑작스런 연결 해제
try {
ws.close(WS_CLOSE_CODE_ABNORMAL_CLOSURE, 'client returned ECONNRESET error');
} catch (closeError) {
// 연결 종료 실패 처리
}
logger.info(errorLog);
} else if (error?.message.includes?.(ERR_WRITE_STREAM_DESTROYED)) {
// 소켓 파괴됨 - 예상 가능한 상황
stats.increment(`${config.SERVICE_NAME}.ws_send.stream_destroyed_errors`, 1, { a
```

```
logger.info(errorLog);

} else {

logger.error(errorLog);

}

}

});

}
```

실시간 데이터 스트리밍 채널별 구현

1. 지원하는 채널 타입

```
export enum Channel {

V4_ACCOUNTS = 'v4_accounts',

V4_CANDLES = 'v4_candles',

V4_MARKETS = 'v4_markets',

V4_ORDERBOOK = 'v4_orderbook',

V4_TRADES = 'v4_trades',

}
```

```
const ALL_CHANNELS = [

Channel.V4_ACCOUNTS,

Channel.V4_CANDLES,
```

```
Channel.V4_MARKETS,

Channel.V4_ORDERBOOK,

Channel.V4_TRADES,

];
```

2. 채널별 구독 검증

```
private validateSubscription(channel: Channel, id?: string): boolean {

switch (channel) {

case (Channel.V4_ACCOUNTS): {

if (id === undefined) return false;

const parts: string[] = id.split('/');

// 형식: {address}/{subaccountNumber}

return parts.length === 2;

}

case (Channel.V4_MARKETS): {

// 마켓 채널은 ID 불필요

return true;

}

case (Channel.V4_ORDERBOOK):

case (Channel.V4_TRADES): {

if (id === undefined) return false;

// 유효한 영구 선물 마켓 티커인지 확인
```

```
return perpetualMarketRefresher.isValidPerpetualMarketTicker(id);

}

case (Channel.V4_CANDLES): {

if (id === undefined) return false;

const { ticker, resolution } = this.parseCandleChannelId(id);

return perpetualMarketRefresher.isValidPerpetualMarketTicker(ticker) &&

resolution !== undefined;

}

default: {

throw new InvalidChannelError(channel);

}

}

}



private parseCandleChannelId(id: string): { ticker: string, resolution?: CandleR

// 형식: {ticker}/{resolution}

const parts: string[] = id.split('/');

const ticker: string = parts[0];

const resolutionString: string = parts[1];

const resolution = Object.values(CandleResolution).find((cr) => cr === resolutio

return { ticker, resolution };

}
```

3. 초기 데이터 제공

```
private async getInitialResponsesForChannels(
channel: Channel,
id?: string,
country?: string,
): Promise<string> {
if (channel === Channel.V4_ACCOUNTS) {
return this.getInitialResponseForSubaccountSubscription(id, country);
}
const endpoint: string | undefined = this.getInitialEndpointForSubscription(chan
if (endpoint === undefined) {
return EMPTY_INITIAL_RESPONSE;
}

return axiosRequest({
method: RequestMethod.GET,
url: endpoint,
timeout: config.INITIAL_GET_TIMEOUT_MS,
headers: { 'cf-ipcountry': country },
transformResponse: (res) => res, // JSON 파싱 비활성화
});
}
```

```
private getInitialEndpointForSubscription(channel: Channel, id?: string): string

switch (channel) {

case (Channel.V4_TRADES): {

return `${COMLINK_URL}/v4/trades/perpetualMarket/${id}`;

}

case (Channel.V4_MARKETS): {

return `${COMLINK_URL}/v4/perpetualMarkets`;

}

case (Channel.V4_ORDERBOOK): {

return `${COMLINK_URL}/v4/orderbooks/perpetualMarket/${id}`;

}

case (Channel.V4_CANDLES): {

const { ticker, resolution } = this.parseCandleChannelId(id!);

return `${COMLINK_URL}/v4/candles/perpetualMarkets/${ticker}?resolution=${resolu

}

default: {

throw new InvalidChannelError(channel);

}

}

}
```

4. 서브계정 채널 특별 처리

```
private async getInitialResponseForSubaccountSubscription(
id?: string,
country?: string,
): Promise<string> {
if (id === undefined) {
throw new Error('Invalid undefined id');
}


try {
const { address, subaccountNumber } = this.parseSubaccountChannelId(id);


// 서브계정 정보와 주문 정보를 병렬로 조회
const [subaccountsResponse, ordersResponse] = await Promise.all([
axiosRequest({
method: RequestMethod.GET,
url: `${COMLINK_URL}/v4/addresses/${address}/subaccountNumber/${subaccountNumber
timeout: config.INITIAL_GET_TIMEOUT_MS,
headers: { 'cf-ipcountry': country },
transformResponse: (res) => res,
}),
axiosRequest({
method: RequestMethod.GET,
```

```
url: `${COMLINK_URL}/v4/orders?address=${address}&subaccountNumber=${subaccountN

timeout: config.INITIAL_GET_TIMEOUT_MS,

headers: { 'cf-ipcountry': country },

transformResponse: (res) => res,

}),

]);



// 응답 병합

return JSON.stringify({

...JSON.parse(subaccountsResponse),

orders: JSON.parse(ordersResponse),

});

} catch (error) {

// 404: 인덱싱되지 않은 서브계정 - 빈 응답 반환

if (error instanceof AxiosSafeServerError && error.status === 404) {

return EMPTY_INITIAL_RESPONSE;

}

// 403: 차단된 주소 - 특별 에러 처리

if (error instanceof AxiosSafeServerError && error.status === 403) {

throw new BlockedError();

}

throw error;

}
```

```
}
```

Kafka 이벤트 수신 및 구독자 필터링

1. Kafka 연결 및 구독

```
export async function connect(): Promise<void> {
await consumer.connect();

// 모든 WebSocket 토픽 구독
await consumer.subscribe({ topics: Object.values(WebsocketTopics) });
}

// WebSocket 토픽 정의
export enum WebsocketTopics {
TO_WEBSOCKETS_SUBACCOUNTS = 'to-websockets-subaccounts',
TO_WEBSOCKETS_TRADES = 'to-websockets-trades',
TO_WEBSOCKETS_MARKETS = 'to-websockets-markets',
TO_WEBSOCKETS_ORDERBOOKS = 'to-websockets-orderbooks',
TO_WEBSOCKETS_CANDLES = 'to-websockets-candles',
}
```

2. 메시지 포워딩 시스템

```
export class MessageForwarder {
private subscriptions: Subscriptions;
private index: Index;
private messageBuffer: { [key: string]: VersionedContents[] };
private batchSending: NodeJS.Timeout;

public start(): void {
// Kafka 메시지 핸들러 등록
updateOnMessageFunction(async (topic, message): Promise<void> => {
return this.onMessage(topic, message);
});

// 배치 메시지 전송 주기적 실행
this.batchSending = setInterval(
() => { this.forwardBatchedMessages(); },
BATCH_SEND_INTERVAL_MS,
);
}

public onMessage(topic: string, message: KafkaMessage): void {
// 큐 대기 시간 측정
stats.timing(
```

```
`${config.SERVICE_NAME}.message_time_in_queue`,
Date.now() - Number(message.timestamp),
config.MESSAGE_FORWARDER_STATSD_SAMPLE_RATE,
{ topic },
);


// 토픽에서 채널 추출
const channel: Channel | undefined = getChannel(topic);
if (channel === undefined) {
logger.error({ message: `Unknown kafka topic: ${topic}` });
return;
}


// Kafka 메시지를 WebSocket 메시지로 변환
let messageToForward: MessageToForward;
try {
messageToForward = getMessageToForward(channel, message);
} catch (error) {
logger.error({
message: 'Failed to get message to forward from kafka message',
error,
});
return;
```

```
}

// 메시지 전달
this.forwardMessage(messageToForward);
}
}
```

3. 채널별 메시지 파싱

```
export function getMessageToForward(
channel: Channel,
message: KafkaMessage,
): MessageToForward {
if (!message || !message.value) {
throw new InvalidForwardMessageError('Got empty kafka message');
}

const messageBinary: Uint8Array = new Uint8Array(message.value);
switch (channel) {
case Channel.V4_ACCOUNTS: {
const subaccountMessage: SubaccountMessage = SubaccountMessage.decode(messageBin
return {
channel,
```

```
id: getSubaccountMessageId(subaccountMessage),

contents: JSON.parse(subaccountMessage.contents),

version: subaccountMessage.version,

};

}

case Channel.V4_CANDLES: {

const candleMessage: CandleMessage = CandleMessage.decode(messageBinary);

return {

channel,

id: getCandleMessageId(candleMessage),

contents: JSON.parse(candleMessage.contents),

version: candleMessage.version,

};

}

case Channel.V4_MARKETS: {

const marketMessage: MarketMessage = MarketMessage.decode(messageBinary);

return {

channel,

id: V4_MARKETS_ID,

contents: JSON.parse(marketMessage.contents),

version: marketMessage.version,

};

}
```

```
case Channel.V4_ORDERBOOK: {

const orderbookMessage: OrderbookMessage = OrderbookMessage.decode(messageBinary

return {

channel,

id: getTickerOrThrow(orderbookMessage.clobPairId),

contents: JSON.parse(orderbookMessage.contents),

version: orderbookMessage.version,

};

}

case Channel.V4_TRADES: {

const tradeMessage: TradeMessage = TradeMessage.decode(messageBinary);

return {

channel,

id: getTickerOrThrow(tradeMessage.clobPairId),

contents: JSON.parse(tradeMessage.contents),

version: tradeMessage.version,

};

}

default:

throw new InvalidForwardMessageError(`Unknown channel: ${channel}`);

}

}
```

4. 구독자 필터링 및 메시지 전달

```
public forwardMessage(message: MessageToForward): void {
// 구독자가 없으면 전달하지 않음
if (!this.subscriptions.subscriptions[message.channel] &&
!this.subscriptions.batchedSubscriptions[message.channel]) {
return;
}

const id: string = message.id;
let subscriptions: SubscriptionInfo[] = [];
// 일반 구독자 조회
if (this.subscriptions.subscriptions[message.channel]) {
subscriptions = this.subscriptions.subscriptions[message.channel][id] || [];
}

// 배치 구독자 처리
if (this.subscriptions.batchedSubscriptions[message.channel] &&
this.subscriptions.batchedSubscriptions[message.channel][message.id]) {
const bufferKey: string = this.getMessageBufferKey(message.channel, message.id);
if (!this.messageBuffer[bufferKey]) {
this.messageBuffer[bufferKey] = [];
}
```

```
this.messageBuffer[bufferKey].push({
contents: message.contents,
version: message.version,
});
}

// 일반 구독자에게 즉시 전송
if (subscriptions.length > 0) {
let numClientsForwarded: number = 0;
subscriptions.forEach((subscription: SubscriptionInfo) => {
if (subscription.pending) {
// 초기 응답 대기 중인 경우 버퍼링
subscription.pendingMessages.push(message);
return;
}
numClientsForwarded += this.forwardToClient(message, subscription.connectionId);
});
stats.increment(`${config.SERVICE_NAME}.forward_to_client_success`, numClientsFo
}
}
```

연결 상태 관리 및 재연결 메커니즘

1. 구독 관리 시스템

```
export class Subscriptions {

// 채널과 ID별 구독자 매핑

public subscriptions: { [channel: string]: { [id: string]: SubscriptionInfo[] }

// 연결 ID별 구독 목록

public subscriptionLists: { [connectionId: string]: Subscription[] };

// 배치 구독자 매핑

public batchedSubscriptions: { [channel: string]: { [id: string]: SubscriptionIn

// 채널별 구독 ID 추적

public subscribedIdsPerChannel: { [channel: string]: Set<string> };


public async subscribe(

ws: WebSocket,

channel: Channel,

connectionId: string,

messageId: number,

id?: string,

batched?: boolean,

country?: string,

): Promise<void> {

// 구독 검증

if (!this.validateSubscription(channel, id)) {
```

```
sendMessage(ws, connectionId, createErrorMessage(

`Invalid subscription id for channel: (${channel}-${id})`,

connectionId,

messageId,

));

return;

}



const subscriptionId: string = this.normalizeSubscriptionId(id);

// Rate limiting 확인

const duration: number = this.subscribeRateLimiter.rateLimit({

connectionId,

key: channel + subscriptionId,

});

if (duration > 0) {

ws.close(WS_CLOSE_CODE_POLICY_VIOLATION, JSON.stringify({ message: 'Rate limited

return;

}



// 초기 응답 조회

let initialResponse: string;

try {

initialResponse = await this.getInitialResponsesForChannels(channel, id, country
```

```
} catch (error) {
let errorMsg = `Internal error, could not fetch data for subscription: ${channel
if (error instanceof BlockedError) {
errorMsg = error.message;
}
sendMessage(ws, connectionId, createErrorMessage(errorMsg, connectionId, message
return;
}


// 구독 정보 저장
const subscription: Subscription = { channel, id: subscriptionId, batched };
if (!this.subscriptionLists[connectionId]) {
this.subscriptionLists[connectionId] = [];
}
// 중복 구독 확인
if (this.subscriptionLists[connectionId].find(
(s) => (s.channel === channel && s.id === subscriptionId)
)) {
sendMessage(ws, connectionId, createErrorMessage(
`Invalid subscribe message: already subscribed (${channel}-${subscriptionId})`,
connectionId,
messageId,
));
```

```
return;
}

this.subscriptionLists[connectionId].push(subscription);
// 구독자 목록에 추가
if (!this.subscriptions[channel]) {
this.subscriptions[channel] = {};
this.subscribedIdsPerChannel[channel] = new Set<string>();
}
if (!this.subscriptions[channel][subscriptionId]) {
this.subscriptions[channel][subscriptionId] = [];
this.subscribedIdsPerChannel[channel].add(subscriptionId);
}

this.subscriptions[channel][subscriptionId].push({
connectionId,
pending: true, // 초기 응답 전송 전까지 대기
pendingMessages: [],
batched,
});

// 초기 응답 전송
sendMessageString(ws, connectionId, createSubscribedMessage(
```

```
channel,
subscriptionId,
initialResponse,
connectionId,
messageId,
));


// 대기 중인 메시지 전송 활성화
for (let i = 0; i < this.subscriptions[channel][subscriptionId].length; i += 1)
if (this.subscriptions[channel][subscriptionId][i].connectionId === connectionId
this.subscriptions[channel][subscriptionId][i].pending = false;
// 대기 중인 메시지들 전송
this.subscriptions[channel][subscriptionId][i].pendingMessages.forEach(
(pendingMessage) => this.forwardMessage!(pendingMessage, connectionId)
);
// 배치 구독인 경우 배치 구독자 목록으로 이동
if (batched) {
this.subscriptions[channel][subscriptionId].splice(i, 1);
if (!this.batchedSubscriptions[channel]) {
this.batchedSubscriptions[channel] = {};
}
if (!this.batchedSubscriptions[channel][subscriptionId]) {
this.batchedSubscriptions[channel][subscriptionId] = [];
```

```
}
this.batchedSubscriptions[channel][subscriptionId].push({
connectionId,
pending: false,
pendingMessages: [],
batched,
});
}
}
}

// 구독 통계 업데이트
stats.gauge(`${config.SERVICE_NAME}.subscriptions.channel_size`,
this.subscribedIdsPerChannel[channel].size, { channel });
}
}
```

2. 배치 메시지 처리

```
public forwardBatchedMessages(): void {
const bufferKeys: string[] = Object.keys(this.messageBuffer);
bufferKeys.forEach((bufferKey: string) => {
const batchedMessages: VersionedContents[] = this.messageBuffer[bufferKey];
```

```
if (batchedMessages.length > 0) {
const { channel, channelString, id } = this.parseMessageBufferKey(bufferKey);
if (!this.subscriptions.batchedSubscriptions[channelString]) {
return;
}
const batchedSubscribers: SubscriptionInfo[] =
this.subscriptions.batchedSubscriptions[channelString][id];
batchedSubscribers.forEach((batchedSubscriber: SubscriptionInfo) => {
// 버전별로 메시지 그룹화
const batchedVersionedMessages = _.groupBy(batchedMessages, (c) => c.version);
_.forEach(batchedVersionedMessages, (msgs, version) => {
try {
this.forwardToClientBatch(
msgs,
batchedSubscriber.connectionId,
channel,
id,
version,
);
} catch (error) {
logger.error({
message: error.message,
connectionId: batchedSubscriber.connectionId,
```

```
error,
});
}
});
});
}
});
// 버퍼 초기화
this.messageBuffer = {};
}

public forwardToClientBatch(
batchedMessages: VersionedContents[],
connectionId: string,
channel: Channel,
id: string,
version: string,
): void {
const connection: Connection = this.index.connections[connectionId];
if (!connection) {
stats.increment(`${config.SERVICE_NAME}.forward_to_client_batch_error`, 1);
this.subscriptions.unsubscribe(connectionId, channel, id);
return;
```

```
}


this.index.connections[connectionId].messageId += 1;

stats.increment(`${config.SERVICE_NAME}.forward_to_client_batch_success`, 1);

sendMessage(connection.ws, connectionId, createChannelBatchDataMessage(

channel,

connectionId,

this.index.connections[connectionId].messageId,

id,

version,

batchedMessages.map((c) => c.contents),

));

}
```

3. 연결 해제 및 정리

```
private disconnect(connectionId: string): void {

if (!this.connections[connectionId]) {

return;

}

try {

// 하트비트 타이머 정리

if (this.connections[connectionId].heartbeat) {
```

```
clearInterval(this.connections[connectionId].heartbeat);

}

// WebSocket 연결 종료

this.connections[connectionId].ws.terminate();



// 구독 데이터 정리

this.subscriptions.remove(connectionId);

this.pingHandler.handleDisconnect(connectionId);

this.invalidMessageHandler.handleDisconnect(connectionId);

// 연결 정보 삭제

delete this.connections[connectionId];

} catch (error) {

logger.error({

message: `Disconnecting threw error: ${error.message}`,

error,

connectionId,

});

}

}



// Subscriptions 클래스의 정리 메서드

public remove(connectionId: string) {

const subscriptionList = this.subscriptionLists[connectionId];
```

```
if (subscriptionList) {
subscriptionList.forEach((subscription) => {
if (subscription.batched) {
// 배치 구독자 목록에서 제거
const idx = this.batchedSubscriptions[subscription.channel][subscription.id]
.findIndex((e: SubscriptionInfo) => e.connectionId === connectionId);
if (idx >= 0) {
this.batchedSubscriptions[subscription.channel][subscription.id].splice(idx, 1);
}
} else {
// 일반 구독자 목록에서 제거
const idx = this.subscriptions[subscription.channel][subscription.id]
.findIndex((e: SubscriptionInfo) => e.connectionId === connectionId);
if (idx >= 0) {
this.subscriptions[subscription.channel][subscription.id].splice(idx, 1);
}
}
});
delete this.subscriptionLists[connectionId];
}

// Rate limiter에서 연결 정보 제거
this.subscribeRateLimiter.removeConnection(connectionId);
```

```
}
```

Rate Limiting 및 보안

1. Rate Limiter 구현

```
export class RateLimiter {
private points: number;
private durationMs: number;
private connectionLimits: { [connectionId: string]: { [key: string]: number } };

constructor(options: { points: number; durationMs: number }) {
this.points = options.points;
this.durationMs = options.durationMs;
this.connectionLimits = {};
}

public rateLimit(options: { connectionId: string; key: string }): number {
if (!config.RATE_LIMIT_ENABLED) {
return 0;
}

const { connectionId, key } = options;
```

```
const now = Date.now();

if (!this.connectionLimits[connectionId]) {

this.connectionLimits[connectionId] = {};

}

const lastRequest = this.connectionLimits[connectionId][key] || 0;

const timeSinceLastRequest = now - lastRequest;

if (timeSinceLastRequest < this.durationMs) {

// Rate limit 위반

return this.durationMs - timeSinceLastRequest;

}

this.connectionLimits[connectionId][key] = now;

return 0;

}


public removeConnection(connectionId: string): void {

delete this.connectionLimits[connectionId];

}

}
```

2. 메시지 타입별 Rate Limiting

```
// 구독 요청 Rate Limiting

const subscribeRateLimiter = new RateLimiter({
```

```
points: config.RATE_LIMIT_SUBSCRIBE_POINTS,

durationMs: config.RATE_LIMIT_SUBSCRIBE_DURATION_MS,

});



// Ping 메시지 Rate Limiting

const pingRateLimiter = new RateLimiter({

points: config.RATE_LIMIT_PING_POINTS,

durationMs: config.RATE_LIMIT_PING_DURATION_MS,

});



// 잘못된 메시지 Rate Limiting

const invalidMessageRateLimiter = new RateLimiter({

points: config.RATE_LIMIT_INVALID_MESSAGE_POINTS,

durationMs: config.RATE_LIMIT_INVALID_MESSAGE_DURATION_MS,

});
```

3. 지역 차단 및 컴플라이언스

```
// 연결 시 국가 코드 추출

this.connections[connectionId] = {

ws,

messageId: 0,

countryCode: getCountry(req), // 헤더에서 국가 코드 추출
```

```
};


// 구독 시 국가별 제한 확인
private async getInitialResponseForSubaccountSubscription(
id?: string,
country?: string,
): Promise<string> {
try {
const response = await axiosRequest({
url: `${COMLINK_URL}/v4/addresses/${address}/subaccountNumber/${subaccountNumber
headers: {
'cf-ipcountry': country, // Cloudflare 국가 헤더 전달
},
});
return response;
} catch (error) {
// 403: 차단된 지역/주소
if (error instanceof AxiosSafeServerError && error.status === 403) {
throw new BlockedError();
}
throw error;
}
```

```
}
```

성능 최적화 및 모니터링

1. 메트릭 수집

```
// 연결 통계
stats.increment(`${config.SERVICE_NAME}.num_connections`, 1);
stats.gauge(`${config.SERVICE_NAME}.num_concurrent_connections`, numConcurrentCo

// 메시지 처리 통계
stats.timing(`${config.SERVICE_NAME}.message_time_in_queue`, queueTime, sampleRa
stats.timing(`${config.SERVICE_NAME}.forward_message`, processingTime, sampleRat

// 구독 통계
stats.gauge(`${config.SERVICE_NAME}.subscriptions.channel_size`, channelSize, {

// 에러 통계
stats.increment(`${config.SERVICE_NAME}.forward_to_client_error`, 1);
stats.increment(`${config.SERVICE_NAME}.ws_send.stream_destroyed_errors`, 1, { a
```

2. 메모리 최적화

```
// 주기적 메시지 버퍼 정리
public forwardBatchedMessages(): void {
// 배치 메시지 처리 후 버퍼 초기화
this.messageBuffer = {};
}

// 연결 해제 시 메모리 정리
private disconnect(connectionId: string): void {
// 모든 관련 데이터 구조에서 연결 정보 제거
this.subscriptions.remove(connectionId);
this.pingHandler.handleDisconnect(connectionId);
this.invalidMessageHandler.handleDisconnect(connectionId);
delete this.connections[connectionId];
}
```

3. 에러 처리 및 복구

```
// 서비스 재시작 시 정리
public async close(): Promise<void> {
Object.keys(this.connections).forEach((connectionId: string) => {
try {
this.connections[connectionId].ws.close(
WS_CLOSE_CODE_SERVICE_RESTART,
```

```
JSON.stringify({ message: 'Service restarting' }),
);
} catch (error) {
logger.error({
message: `Failed to close connection: ${error.message}`,
connectionId,
error,
});
}
});
await this.wss.close();
}

// 메시지 처리 에러 복구
private onMessage(connectionId: string, message: WebSocket.Data): void {
try {
// 메시지 처리 로직
} catch (error) {
logger.error({
message: `Failed to handle message: ${error.message}`,
error,
connectionId,
});
```

```
// 에러 메시지 전송
sendMessage(
this.connections[connectionId].ws,
connectionId,
createErrorMessage('Internal error', connectionId, this.connections[connectionId
);
}
}
```

결론

Socks 서비스는 dYdX v4의 실시간 데이터 스트리밍을 담당하는 핵심 WebSocket 서버로, 확장 가능한 연결 관리, 효율적인 메시지 라우팅, 강력한 보안 및 Rate Limiting 기능을 제공합니다. Kafka와의 통합을 통해 실시간 이벤트를 안정적으로 전달하며, 배치 처리와 채널별 필터링을 통해 높은 성능과 유연성을 달성하고 있습니다.